

C++ Practical Problem Handbook

Problem Definitions, Complete Source Code with using namespace std; & Terminal Outputs

Program 1: Symmetric Alphabet Triangle Pattern

ABCDTRI1.CPP

Problem: Read the number of rows n and output a mirrored character pyramid pattern where letters increment towards the center and decrement after a dynamic gap.

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    int n;
    cout << "How many rows are there? ";
    cin >> n;

    for (int i = n; i >= 1; i--) {
        char ch = 'A';
        for (int j = 1; j <= i; j++) cout << setw(2) << ch++;
        for (int s = 0; s < 2 * (n - i); s++) cout << " ";
        ch--;
        for (int j = 1; j <= i; j++) cout << setw(2) << ch--;
        cout << "\n";
    }
    return 0;
}
```

Sample Output: How many rows are there? 4 A B C D D C B A A B C C B A A B B A A A

Program 2: Character Span Range Display

A-E.CPP

Problem: Input a three-character range string formatted as Start-End (e.g., A-E) and print all characters in the sequence with cyclical alphabetical wrap-around.

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string str;
    cout << "Enter character range: ";
    cin >> str;

    if (str.length() >= 3) {
        char start = str[0], end = str[2], cur = start;
        while (cur != end) {
            cout << cur << " ";
            cur++;
            if (cur > 'Z') cur = 'A';
        }
        cout << end << "\n";
    }
    return 0;
}
```

Sample Output: Enter character range: A-E A B C D E

Program 3: Array Sorting State Classifier

ASCDES.CPP

Problem: Determine if an array of integers is in Ascending Order, Descending Order, Unsorted, or if All Elements are Equal in a single traversal.

```
#include <iostream>
using namespace std;

int main() {
    int a[50], no;
    char ans = 'E'; // 'E': Equal, 'A': Ascending, 'D': Descending, 'N': Not Sorted
    cout << "Enter no of Elements: ";
    cin >> no;
    for (int i = 0; i < no; i++) {
        cout << "Enter a[" << i << "] = ";
        cin >> a[i];
    }
    for (int i = 0; i < no - 1; i++) {
        if (a[i] == a[i + 1]) continue;
        if (a[i] < a[i + 1]) {
            if (ans == 'D') { ans = 'N'; break; }
            ans = 'A';
        } else {
            if (ans == 'A') { ans = 'N'; break; }
            ans = 'D';
        }
    }
    if (ans == 'A') cout << "Array sorted in Ascending order\n";
    else if (ans == 'D') cout << "Array sorted in Descending Order\n";
    else if (ans == 'N') cout << "Not Sorted\n";
    else cout << "All elements are equal\n";
    return 0;
}
```

Sample Output: Enter no of Elements: 5 Enter a[0] = 12 | a[1] = 25 | a[2] = 40 | a[3] = 55 | a[4] = 80 Array sorted in Ascending order

Program 4: Calculate Next Calendar Date

DATE_NEXT.CPP

Problem: Input date (day, month, year) and compute the subsequent date considering month length, leap years (Feb 29), and year rollovers.

```
#include <iostream>
using namespace std;

bool isLeapYear(int y) {
    return (y % 4 == 0 && (y % 100 != 0 || y % 400 == 0));
}

int main() {
    int d, m, y;
    int month[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
    cout << "Enter Date, Month & Year: ";
    cin >> d >> m >> y;

    if (m == 2 && isLeapYear(y)) month[1] = 29;

    if (d < month[m - 1]) d++;
    else if (m < 12) { d = 1; m++; }
    else { d = 1; m = 1; y++; }

    cout << "The new date is: " << d << "-" << m << "-" << y << "\n";
    return 0;
}
```

Sample Output: Enter Date, Month & Year: 28 2 2024 The new date is: 29-2-2024

Program 5: Day of Week & Elapsed Date Difference

DATEDIF2.CPP

Problem: Compute total elapsed days from standard reference date (01-01-1900 Monday) to any given target date, then identify the day name of the week.

```
#include <iostream>
#include <string>
using namespace std;

bool isLeapYear(int y) { return (y % 4 == 0 && (y % 100 != 0 || y % 400 == 0)); }

int main() {
    int d2, m2, y2;
    int monthDays[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
    string days[] = {"Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"};

    cout << "Enter Ending Date (DD MM YYYY): ";
    cin >> d2 >> m2 >> y2;

    long totalDays = 0;
    for (int y = 1900; y < y2; y++) totalDays += isLeapYear(y) ? 366 : 365;
    for (int m = 1; m < m2; m++) {
        if (m == 2 && isLeapYear(y2)) totalDays += 29;
        else totalDays += monthDays[m - 1];
    }
    totalDays += (d2 - 1);

    cout << "Total Days are: " << totalDays << "\n";
    cout << "For Given Date Day is: " << days[totalDays % 7] << "\n";
    return 0;
}
```

Sample Output: Enter Ending Date (DD MM YYYY): 1 1 2000 Total Days are: 36524 For Given Date Day is: Saturday

Program 6: Calculate Previous Calendar Date

DTOMY.CPP

Problem: Determine the previous calendar day from a given date, handling end-of-month backward rollovers and leap year checks for March 1st.

```
#include <iostream>
using namespace std;

bool isLeapYear(int y) { return (y % 4 == 0 && (y % 100 != 0 || y % 400 == 0)); }

int main() {
    int d, m, y;
    int month[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
    cout << "Enter Date, Month & Year: ";
    cin >> d >> m >> y;

    if (d > 1) d--;
    else if (m > 1) {
        m--;
        d = (m == 2 && isLeapYear(y)) ? 29 : month[m - 1];
    } else {
        d = 31; m = 12; y--;
    }

    cout << "The previous date is: " << d << "-" << m << "-" << y << "\n";
    return 0;
}
```

Sample Output: Enter Date, Month & Year: 1 3 2024 The previous date is: 29-2-2024

Problem: Extract and display the word of maximum length from a user-supplied sentence containing multiple arbitrary spaces.

```
#include <iostream>
#include <string>
#include <sstream>
using namespace std;

int main() {
    string str, word, maxWord = "";
    cout << "Enter the String: ";
    getline(cin, str);

    stringstream ss(str);
    while (ss >> word) {
        if (word.length() > maxWord.length()) {
            maxWord = word;
        }
    }
    cout << "The maximum length word is: " << maxWord
        << " (Length: " << maxWord.length() << ")\n";
    return 0;
}
```

Sample Output: Enter the String: Bhavnagar University Computer Science The maximum length word is: University (Length: 10)

Problem: Calculate factorial of n and evaluate the alternating factorial series: $1/1! - 2/2! + 3/3! - 4/4! + \dots + n/n!$.

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    int n;
    cout << "Enter n to calculate factorial & series: ";
    cin >> n;

    long fact = 1;
    for (int i = 1; i <= n; i++) fact *= i;
    cout << "Factorial Value is = " << fact << "\n";

    float sum = 0.0f;
    fact = 1;
    int sign = 1;
    cout << "Series: ";
    for (int i = 1; i <= n; i++) {
        fact *= i;
        sum += (float)(i * sign) / fact;
        cout << i << "/" << fact << (sign > 0 ? " - " : " + ");
        sign = -sign;
    }
    cout << "= " << fixed << setprecision(4) << sum << "\n";
    return 0;
}
```

Sample Output: Enter n to calculate factorial & series: 4 Factorial Value is = 24 Series: 1/1 - 2/2 + 3/6 - 4/24 = 0.3333

Program 9: Mathematical Sequence Generation

PRIL_PR5.CPP

Problem: Generate and format n terms of the quadratic series where the k -th term is $(k+1)^2 - k$, printing exactly 10 values per row.

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    int n;
    cout << "Enter no of terms: ";
    cin >> n;

    for (int i = 1; i <= n; i++) {
        long term = (1L * (i + 1) * (i + 1)) - i;
        cout << setw(5) << term << (i == n ? "." : ", ");
        if (i % 10 == 0) cout << "\n";
    }
    cout << "\n";
    return 0;
}
```

Sample Output: Enter no of terms: 12 3, 7, 13, 21, 31, 43, 57, 73, 91, 111, 133, 157.

Program 10: Fibonacci and Lucas Series Generators

PRIL_PR6.CPP

Problem: Display the first n terms of the Fibonacci Series (0, 1, 1, 2, 3...) and the Lucas Series (1, 3, 4, 7, 11...) using iterative variable shifting.

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    int n;
    cout << "Enter no of terms: ";
    cin >> n;

    cout << "Fibonacci Series:\n";
    int a = 0, b = 1;
    for (int i = 1; i <= n; i++) {
        cout << setw(5) << a;
        int next = a + b;
        a = b; b = next;
    }

    cout << "\nLucas Series:\n";
    int l1 = 1, l2 = 3;
    for (int i = 1; i <= n; i++) {
        cout << setw(5) << l1;
        l2 = l1 + l2;
        l1 = l2 - l1;
    }
    cout << "\n";
    return 0;
}
```

Sample Output: Enter no of terms: 7 Fibonacci Series: 0 1 1 2 3 5 8 Lucas Series: 1 3 4 7 11 18 29

Program 11: Automorphic Number Range Search

PRIL_PR9.CPP

Problem: Identify and print all automorphic numbers up to a specified maximum range n (numbers whose square ends in the same digits as the number itself).

```
#include <iostream>
using namespace std;

bool isAutomorphic(long n) {
    long sq = n * n, temp = n;
    while (temp > 0) {
        if (temp % 10 != sq % 10) return false;
        temp /= 10;
        sq /= 10;
    }
    return true;
}

int main() {
    long maxLimit;
    cout << "Enter Maximum Number: ";
    cin >> maxLimit;
    for (long i = 1; i <= maxLimit; i++) {
        if (isAutomorphic(i)) {
            cout << "Square of " << i << " ==> " << (i * i) << "\n";
        }
    }
    return 0;
}
```

Sample Output: Enter Maximum Number: 1000 Square of 1 ==> 1 Square of 5 ==> 25 Square of 6 ==> 36 Square of 25 ==> 625 Square of 76 ==> 5776 Square of 376 ==> 141376 Square of 625 ==> 390625

Program 12: Right-Justified Number Triangle

TRIAN2.CPP

Problem: Accept the number of rows n and output a right-aligned triangle with column-spaced incrementing digits.

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    int n;
    cout << "Enter number of rows: ";
    cin >> n;

    for (int i = 1; i <= n; i++) {
        for (int s = 1; s <= (n - i); s++) cout << " ";
        for (int j = 1; j <= i; j++) cout << setw(3) << j;
        cout << "\n";
    }
    return 0;
}
```

Sample Output: Enter number of rows: 5 1 1 2 1 2 3 1 2 3 4 1 2 3 4 5